

BIG DATA · LAB 03 · SEMINAR 75-90 PHÚT

MapReduce & Spark trên 129 nghìn đơn hàng Amazon

4 bài toán · cách nghĩ, cách làm, cách tự kiểm tra kết quả

Dữ liệu: Amazon Sale Report.csv · 128.975 dòng · 31/03 → 29/06/2022

80 phút, 4 bài, 1 thông điệp

PHÚT	NỘI DUNG	CÔNG CỤ
0–10	Dữ liệu và 5 cái bẫy	—
10–32	Bài 1-1 · Cửa sổ trượt	MapReduce
32–42	Bài 1-2 · Median variety	MapReduce
42–60	Bài 2-1 · Tỷ lệ huỷ (kết quả 0!)	Spark DataFrame
60–74	Bài 2-2 · Percentile động	Spark DataFrame
74–80	Nộp bài + chốt	—

Thông điệp xuyên suốt

Điểm không nằm ở con số cuối. **1,5 / 2,5 điểm mỗi bài nằm ở lập luận**: đọc đề đúng, chốt chỗ mập mờ, và chứng minh được kết quả của mình.

Bài 1 = MapReduce thật

Nộp code mapper / reducer. SQL nếu có chỉ để **dò lại đáp số**, không phải bài làm.

Biết dữ liệu trước, code sau: 5 cái bẫy

① "Shipped" không phải một giá trị

Status có 13 giá trị, **10** cái chứa chữ shipped. So sánh =
'Shipped' chỉ bắt 77.804 dòng; phải dùng chứa
'shipped' → 109.592 dòng.

② Tên bang viết đủ kiểu

69 cách viết → chuẩn hoá UPPER(TRIM(...)) còn **46**
bang. Không làm thì Delhi / DELHI / delhi thành 3 bang.

③ Ngày là MM-DD-YY

04-30-22 = 30/04/2022. Parse nhầm DD-MM là hỏng cả
bài chữa số.

④ Size không sort theo chữ được

Theo bảng chữ cái thì 3XL < XXL — sai! Phải tự đánh số:
XS=1 ... XXL=6, 3XL=7... "Ít nhất XXL" = rank ≥ 6.

⑤ NULL ở khắp nơi

12.807 dòng qty = 0, 7.795 dòng Amount null. So sánh
với null trả về null → dùng coalesce tường minh.

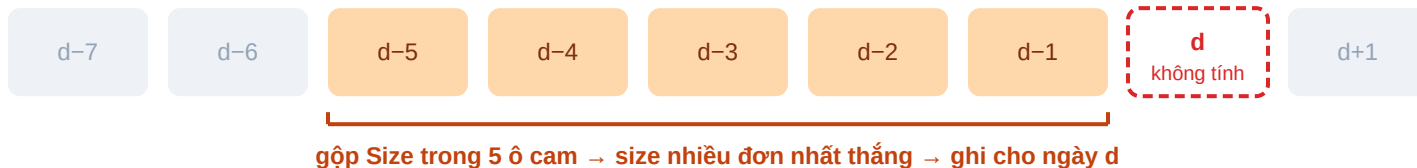
Tự kiểm: đọc CSV xong, mọi dòng phải có đúng **24 cột** (cột
promotion-ids chứa dấu phẩy bên trong — cần parser có
quote).

Đề hỏi gì? Một câu thôi

Mỗi bang, mỗi ngày d: size nào bán chạy nhất trong những ngày TRƯỚC ngày d?

Bang bán > 10.000 đơn → nhìn lại 5 ngày. Bang còn lại → nhìn lại 10 ngày.

Cửa sổ của ngày d ($L = 5$): lấy các đơn từ d-5 đến d-1 — KHÔNG lấy ngày d

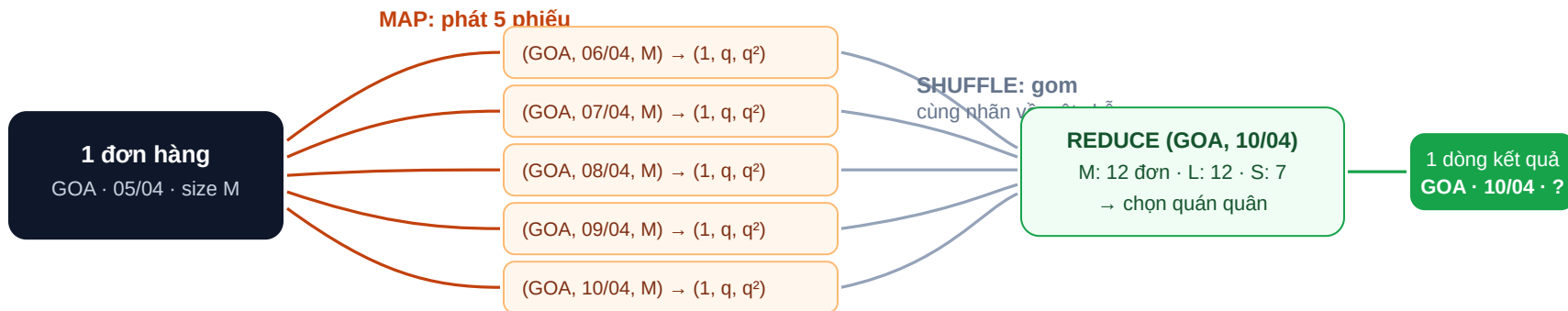


"Đã bán" = Status chứa shipped VÀ Qty > 0 · Chỉ 2/46 bang vượt 10.000 đơn: MAHARASHTRA (19.103), KARNATAKA (14.950)

Lỗi hay gặp nhất: viết cửa sổ thừa ngày d, hoặc chỉ lấy 4 ngày (off-by-one). Chạy vẫn trơn tru, số sai một cách kín đáo. Ngày cuối có đơn là 29/06 nhưng phải xuất kết quả tới **09/07** (= 29/06 + 10).

MapReduce giải bài này thế nào

Lật ngược câu hỏi: đừng hỏi "ngày d cần dữ liệu nào" — hãy hỏi "đơn này sẽ được dùng cho những ngày nào".



Mỗi phiếu mang bộ ba (1, q, q²) — cộng dồn được → ra số đơn, trung bình VÀ phương sai. Nhờ đó Combiner gộp ngay tại máy map.

Cần một job phụ (Job 0) chạy trước để đếm tổng đơn từng bang → biết bang nào cửa sổ 5, bang nào 10. Bảng 46 dòng này phát cho mọi mapper qua Distributed Cache.

Hai size bằng điểm nhau — ai thắng?

514

trên 3.696 nhóm có ≥ 2 size đồng hạng nhất

1/7

kết quả do luật phá hoà quyết định — không phải chi tiết phụ

Luật 1 — giá ổn định hơn thắng

M: 12 đơn, giá quanh quần 300–320 ₹ (dao động ít)

L: 12 đơn, giá nhảy 100 → 900 ₹ (dao động dữ)

→ **M thắng**. Nhu cầu đều đặn đáng tin hơn một cú trúng đơn sỉ. "Dao động" đo bằng phương sai — **chia N** (cả cửa sổ là tổng thể, và công thức chia N mới cộng dồn được cho Combiner).

Điểm mập mờ phải ghi vào báo cáo

Đề viết "*variance of the purchased amount*" — cột `Amount` (tiền) hay `Qty` (lượng)? Hai cách hiểu cho **204/3.696 dòng (5,5%) khác nhau**.

Chọn gì cũng được, **miễn ghi rõ và có lý do**. Khuyến nghị: `Amount` (khớp tên cột).

Luật 2 — vẫn hoà: theo bảng chữ cái

Không có ý nghĩa nghiệp vụ. Chỉ để ai chạy cũng ra đúng một đáp án — chấm bài `diff` được.

Mẹo cài đặt: cấu hình secondary sort để reducer nhận các size đã xếp A → Z sẵn → chỉ cần một biến `best`, đôi khi thực sự lớn hơn (dùng `>`, không `>=`) là luật 2 tự đúng. Chi tiết ở phụ lục A1.

Đề bắt so sánh "naive vs optimized" — trả lời thế này

Đề viết (phần yêu cầu report): *"Time complexity: $O(n \times w)$ naive vs optimized · Shuffle complexity analysis"*

Dịch: đưa ra **hai cách làm** — ngây thơ và tối ưu — rồi so **lượng phiếu đẩy qua mạng** (shuffle, bước chậm nhất: mạng chậm hơn CPU hàng trăm lần).

Lời giải: cùng một đáp án, ba mức gửi phiếu. Cách 1 là "naive" của đề, cách 2+3 là "optimized":

Cách 1 — phát phiếu cho TỪNG ngày ("naive")

1 đơn (cửa sổ 10 ngày) = 10 phiếu

925.395 phiếu — gấp 8,4 lần số đơn

Cách 2 — mảng hiệu ("optimized")

1 đơn = 2 phiếu ĐẦU/CUỐI: "+1 từ 06/04" · "-1 từ 16/04"

219.132 — gấp 2

Cách 3 — thêm Combiner: gộp trước khi gửi

200 phiếu cùng nhãn trên một máy → 1 phiếu ghi "200"

27.134 — còn 1/4 số đơn, nén 34 lần so với cách 1

Đáp án cuối của ba cách **giống hệt nhau** — khác nhau ở hoá đơn tiền mạng (CPU cũng ngang nhau).

Câu ăn điểm: vì sao cách 1 "tệ" mà vẫn nộp được?

SỐ NHÃN khác nhau có trần cố định: 46 bang × 101 ngày × 11 size ≈ **51 nghìn nhãn là kịch**. Dữ liệu tăng 100 lần thì phiếu tăng, nhưng sau khi Combiner gộp theo nhãn, đầu ra **không bao giờ vượt trần**.

Kết quả đúng trông như thế nào

3.696

dòng (bang × ngày)

09/07

ngày cuối cùng phải có (29/06 + 10)

M

size tăng nhiều nhất (1.299 lần), rồi L, XL...

3 phép kiểm tra 30 giây

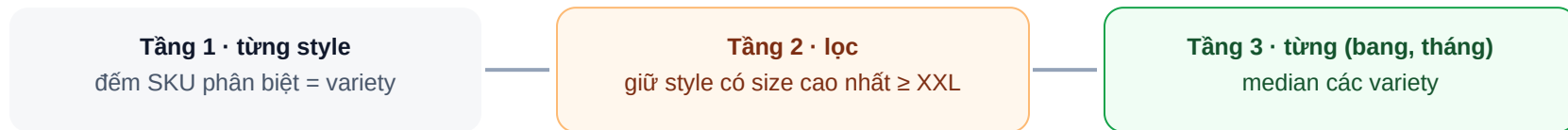
- ▶ Ngày lớn nhất trong file = **2022-07-09**?
- ▶ Chỉ MAHARASHTRA, KARNATAKA có cửa sổ 5?
- ▶ Không có dòng nào cho ngày 31/03 (chưa có quá khứ)?

Trong báo cáo cần có

- ▶ Thiết kế khoá + secondary sort
- ▶ Bảng shuffle 3 phương án (slide trước)
- ▶ Chốt Amount vs Qty + lý do

Median variety: bóc đề thành 3 tầng

Variety của một style = số SKU khác nhau của style đó trong một (bang, tháng).
Chỉ xét style **đã từng bán size $\geq$ XXL**. Đáp số = **median của variety** cho mỗi (bang, tháng).



MapReduce: Job 1 khoá (tháng, bang, style, sku) — đếm SKU bằng cách đếm lần đối khoá, không cần HashSet · Job 2 khoá (tháng, bang) — sort rồi lấy giữa

Bẫy chết người: sort size theo chữ

Theo bảng chữ cái, 3XL đứng TRƯỚC XXL → "size lớn nhất" sai hoàn toàn. Phải dùng thang số: XS=1, S=2, M=3, L=4, XL=5, **XXL=6**, 3XL=7... Sort chữ cho 18.096 dòng "≥ XXL"; thang số cho **34.627** — lệch gần gấp đôi.

Một câu đề, hai cách hiểu — lệch 31% kết quả

"Style đã từng bán XXL" — xét ở đâu?

Trong nhóm: style phải bán XXL *tại chính bang đó, tháng đó*.

Toàn cục: style bán XXL ở bất kỳ đâu là đủ điều kiện ở mọi nơi.

Đo thật: hai cách lệch **40/128 nhóm (31%)**. Đề không nói rõ → **chọn một, ghi rõ, giải thích**. Đề đang nói về "a specific time interval and region" → nghiêng về *trong nhóm*; file đáp án của giảng viên lại làm *toàn cục*. Nêu cả hai trong báo cáo là an toàn nhất.

Median khi số phần tử chẵn

Lấy trung bình 2 phần tử giữa (khớp numpy/Spark). 3/128 nhóm ra giá trị .5.

128

dòng kết quả (tháng × bang), theo cách "trong nhóm"

Kiểm tra nhanh

- ▶ Tháng 3 chỉ có ngày 31/03 → 16 nhóm, median đều 1,0
- ▶ MAHARASHTRA 04/2022: 647 style, median 4,0
- ▶ Nhóm to nhất 647 phần tử → sort trong reducer thoải mái

PHẦN B · PHÚT 42-74

Spark Structured APIs

Bài 2-1 và 2-2. Ràng buộc của đề: **chỉ dùng DataFrame API**, không viết SQL dạng chuỗi.
Và một bất ngờ: bài 2-1 có đáp án bằng **0** — cố tình.

Dịch đề từng dòng trước, code sau

ĐỀ VIẾT	NGHĨA LÀ	CON SỐ THẬT
"cancelled orders of Standard service level"	Status chứa Cancelled và ship-service-level = Standard	6.909 đơn
"promotion's active period = first → last order date; valid if ≥ 2 days"	KM không có sẵn ngày hiệu lực — phải tự suy: ngày đầu và ngày cuối mã đó xuất hiện trong dữ liệu. Sống ≥ 2 ngày mới "hợp lệ"	185 / 284 mã
"at least 3 temporally-valid promotions (including Amazon's)"	đơn chứa ≥ 3 mã thuộc nhóm hợp lệ — mã Amazon vẫn đếm	điều kiện quyết định
"Amount < average of the state's Merchant-fulfilled Shipped orders"	giá đơn nhỏ hơn mức trung bình của bang (chỉ tính đơn Merchant + Courier Shipped)	40 ngưỡng bang
"percentage per city"	gộp theo thành phố , ra % đơn thoả các điều kiện trên	1.435 t.phố

Ví dụ dòng 2: mã X xuất hiện lần đầu 25/04, lần cuối 30/04 → sống 5 ngày → **hợp lệ**. Mã Y chỉ xuất hiện đúng ngày 01/05 → sống 0 ngày → **loại**. Suy tuổi thọ từ dữ liệu = một `groupBy(mã KM)` lấy min/max ngày — đây là job phụ đầu tiên của bài.

Ghép 5 dòng vừa dịch thành 4 khối code

	KHỐI	LÀM GÌ	RA
A	Khuyến mãi	mỗi mã KM: số ngày hoạt động = ngày cuối – ngày đầu; giữ mã ≥ 2 ngày	284 → 185 mã
B	Ngưỡng bang	giá trung bình đơn Merchant + Courier "Shipped", theo bang	40 dòng
C	Tập đơn	đơn Standard + Cancelled; join A đếm mã hợp lệ, join B so ngưỡng	6.909 đơn
D	Gộp	theo thành phố : % đơn thoả (≥ 3 mã hợp lệ VÀ Amount < ngưỡng)	1.435 t.phố

Bẫy địa lý

Ngưỡng tính theo **bang**, kết quả gộp theo **thành phố** — hai mức khác nhau trong cùng một câu đề. Rất dễ đọc lướt thành một.

Bẫy nội dung

270/284 mã KM do Amazon phát hành — đề dẫn **giữ lại** ("all promotions, including those issued by Amazon"). Ai quen đề cũ mà loại đi là sai.

Bẫy join

Phải **LEFT join**: đơn không có KM vẫn phải nằm ở mẫu số. INNER join làm mất đơn → phần trăm sai.

Kết quả là 0% — và đó là đáp án đúng

Vì sao bằng 0 — chuỗi suy luận 3 bước

- ▶ Toàn bộ **6.909** đơn Cancelled + Standard đều có `promotion-ids` **rỗng**
- ▶ Nói ra mọi đơn Cancelled (18.332): chỉ 295 đơn có KM, và mỗi đơn có **đúng 1 mã**
- ▶ Điều kiện " ≥ 3 mã" không bao giờ chạm được → tử số = 0 ở mọi thành phố

Đã thử 4 cách hiểu chữ "cancelled", thử gom theo Order ID, thử bỏ từng bộ lọc — **tất cả vẫn ra 0**. Kiểm chứng bằng 3 công cụ độc lập. File đáp án của giảng viên (pipeline khác hẳn) cũng ra 0.

Viết gì vào báo cáo

Nộp file 0% **kèm chứng minh** như cột trái → được cộng điểm lập luận.

Đừng làm điều này

Tự nói điều kiện (hạ ≥ 3 xuống $\geq 1...$) để "ra số đẹp" → **sai đề**, dù số trông hợp lý hơn.

Mẹo khi pipeline ra rỗng

Hạ từng điều kiện một. Số dòng tăng dần hợp lý → code đúng, dữ liệu rỗng thật. Vẫn 0 ở mọi mức → lỗi ở phép join.

explain(true) in ra cái gì? Học đọc trong 3 phút

Spark không chạy từng lệnh ngay. Nó gom cả chuỗi lệnh, soạn một **kế hoạch chạy**, rồi mới thực thi. `explain(true)` in kế hoạch đó ra. Đề bắt ta đọc nó — và chỉ cần nhận mặt **hai từ khoá**:

① Exchange = một lần shuffle

Dữ liệu bị **chia bài lại giữa các máy theo khoá** — y hệt shuffle của MapReduce sáng nay, và cũng đắt y hệt (mạng + đĩa).

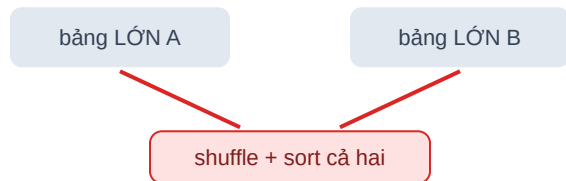
Quy tắc đếm nhanh: mỗi `groupBy` theo một khoá mới = một Exchange. Nhìn code đếm được số shuffle trước cả khi chạy.

② Tên join = chiến lược join

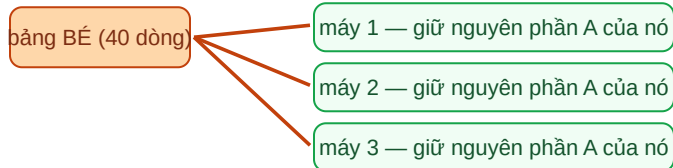
SortMergeJoin: shuffle **cả hai bảng** theo khoá, sort từng bên, rồi trộn. Chịu được 2 bảng lớn — nhưng tốn 2 shuffle + 2 sort.

BroadcastHashJoin: một bảng đủ bé (< 10 MB) → gửi **nguyên bảng** tới mọi máy; bảng lớn đứng yên tại chỗ. Không shuffle bảng lớn, không sort — rẻ hơn hẳn.

SortMergeJoin: hai bảng cùng phải di chuyển



BroadcastHashJoin: bảng bé tự tìm đến bảng lớn



Áp vào bài 2-1: đối chiếu code với plan

Đếm shuffle từ code: 4 lần groupBy

- ▶ groupBy **mã KM** → tính tuổi thọ từng mã
 - ▶ groupBy **mã đơn** → đếm mã hợp lệ mỗi đơn
 - ▶ groupBy **bang** → ngưỡng giá trung bình
 - ▶ groupBy **thành phố** → kết quả cuối
- plan phải có đúng **4 Exchange**. Khớp!

Vì sao 3 join đều là Broadcast?

Ba bảng đi join đều tí hon: 185 mã hợp lệ, bảng đếm-mã, 40 ngưỡng bang → Spark chọn BroadcastHashJoin cả 3, khởi shuffle bảng đơn hàng 129 nghìn dòng.

ĐỌC TỪ PLAN	MẶC ĐỊNH	ÉP TẮT BROADCAST
Kiểu join	3 BroadcastHashJoin	3 SortMergeJoin
Exchange	4	7
Node Sort	0	6

Cách trình bày ăn điểm: chạy 2 lần — mặc định và `autoBroadcastJoinThreshold = -1` — chụp cả hai plan, chỉ vào từng khác biệt của bảng này.

Cảnh báo AQE: Spark 3+ bật AQE mặc định → explain in kế hoạch **tĩnh**; kế hoạch thật xem ở Spark UI, tab SQL.

Bài 2-2: dịch đề từng dòng

ĐỀ VIẾT	NGHĨA LÀ	CON SỐ THẬT
"for each SKU within each month"	chia dữ liệu thành các nhóm (SKU, tháng) — mọi phép tính đều làm riêng trong từng nhóm	16.486 nhóm
"number of promotions per order — ALL promotions"	đếm số mã trong ô <code>promotion-ids</code> của từng đơn, kể cả mã Amazon ; ô trống = 0	0 → 26 mã/đơn
"dynamic percentile thresholds P90, P80"	mốc lọc tự co giãn theo nhóm — không phải con số cứng chung cho cả bài. Mỗi nhóm tự tính P90 của chính nó	mỗi nhóm một ngưỡng
"orders at or above each threshold"	trong nhóm, giữ những đơn có số mã $\geq$ ngưỡng — tức nhóm đơn "được khuyến mãi dày nhất"	~10% / ~20% số đơn
"standard deviation of Amount, ddof = 0"	đo độ phân tán giá của các đơn vừa giữ — chia N. Câu hỏi nghiệp vụ: đơn ôm nhiều mã KM có giá loạn không?	nhóm < 2 đơn → 0

Percentile là gì — nói bằng lời thường

Xếp cả nhóm tăng dần theo số mã KM. **P90 là con số đứng ở mốc 90% của hàng** — 90% số đơn có số mã $\leq$ nó. P90 = "ngưỡng để lọt vào top 10% dày mã nhất của nhóm". P80 tương tự với top 20%. Vì mỗi nhóm xếp hàng riêng, ngưỡng mỗi nhóm mỗi khác — đó là chữ **"dynamic"** trong đề.

Bài 2-2 đề nói gì — làm tay tròn một nhóm

Lấy một nhóm (SKU, tháng) có 10 đơn. Mỗi đơn đếm số mã KM trong `promotion-ids` (kể cả mã Amazon):



Số mã mỗi đơn, đã xếp tăng dần — hai ô cam là 2 đơn "chịu khuyến mãi dày nhất" của nhóm

BƯỚC	LÀM GÌ	TRÊN NHÓM NÀY
①②	đếm mã mỗi đơn, gom theo (SKU, tháng)	10 đơn như trên
③	ngưỡng P90 = mốc chặn 10% đơn "dày mã" nhất, tính trong chính nhóm	P90 = 5
④	giữ đơn có số mã $\geq$ ngưỡng	2 đơn (499 và 599)
⑤	độ lệch chuẩn của Amount các đơn đó, chia N	sd = 50 → một dòng kết quả

Lặp y hệt với P80 (mốc 20%). Và làm cho **cả 16.486 nhóm cùng lúc** — chỗ đó Spark mới vào cuộc.

Từ làm tay sang code: "dán ngưỡng vào từng dòng"

Lúc làm tay, mắt ta nhìn được **cả nhóm cùng lúc**. Spark thì ngược lại: mỗi dòng chỉ biết mình nó. Mà ngưỡng P90 là con số **của cả nhóm**, còn phép so " $\geq$ ngưỡng?" xảy ra **ở từng đơn**. Vậy code luôn có 2 nhịp:

Bảng ĐƠN (mỗi đơn 1 dòng)

đơn #8 · SKU A · 04/22 · 3 mã

đơn #9 · SKU A · 04/22 · 5 mã

đơn #10 · SKU A · 04/22 · 9 mã

... 129 nghìn dòng ...

NHỊP 1

Bảng NGƯỠNG — mỗi nhóm 1 dòng
(SKU A, 04/22) → P90 = 5

groupBy(SKU, tháng)

Sau khi dán: mỗi đơn tự so được

đơn #8 · 3 mã · ngưỡng 5 → $3 \geq 5$? LOẠI

đơn #9 · 5 mã · ngưỡng 5 → GIỮ (giá 499)

đơn #10 · 9 mã · ngưỡng 5 → GIỮ (giá 599)

groupBy lần cuối → sd(499, 599) = 50 ✓

(Window function = cùng ý tưởng, Spark tự "dán" giúp)

NHỊP 2: join ngược theo (SKU, tháng) — "dán" ngưỡng của nhóm vào từng dòng

Bắt ① — chọn nhầm hàm sd

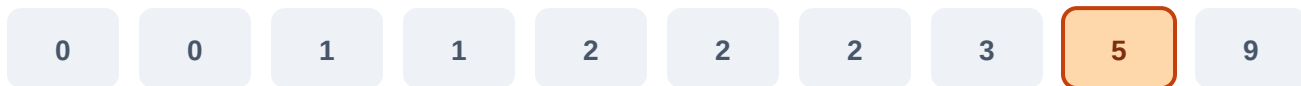
Với 2 đơn 499 và 599: `stddev_pop` (chia N, đề yêu cầu) = **50**. Hàm `stddev` mặc định (chia N-1) = **70,7** — lệch 41%.
Thiếu đúng 4 chữ `_pop`, sai hàng nghìn nhóm.

Bắt ② — nhóm sau lọc còn quá ít đơn

Còn 1 đơn → sd = 0.0, ổn. Còn **0 đơn có giá** (Amount null hết) → sd = **null**, cột kết quả lủng lỗ → phải `coalesce` về 0.0 như đề dẫn "nhóm < 2 đơn → 0".

"Exact" và "approx" khác nhau ở đâu — cùng 10 đơn đó

Nhóm 10 đơn, số mã KM đã xếp: cần P90 (mốc 90%)



nearest-rank ($\approx$ approx của Spark): lấy phần tử thứ $[0,9 \times 10] = 9 \rightarrow$ ngưỡng 5 $\rightarrow$ giữ 2 đơn

nội suy tuyến tính (numpy): $5 + 0,1 \times (9 - 5) = 5,4 \rightarrow$ chỉ giữ 1 đơn $\rightarrow$ sd bị ép về 0!

Số mã KM là **số nguyên nhỏ, trùng nhau rất nhiều** $\rightarrow$ ngưỡng chênh 0,4 cũng quét sạch cả cụm đơn. **Chốt định nghĩa ngay đầu báo cáo** rồi mới so sánh.

Đo thật trên 16.486 nhóm: **43,5%** nhóm lệch ngưỡng nhưng chỉ **0,8%** lệch kết quả cuối. Bài học: lệch ở bước giữa $\neq$ lệch ở đáp số. Tốc độ: exact chậm hơn $\sim 5\text{--}30\%$ (chạy ≥ 5 lần, báo cả mean lẫn sd — JVM warm-up nhiều lần).

Câu hỏi repartition: trả lời bằng số đo, không đoán

16.486

nhóm (SKU, tháng)

426

nhóm lớn nhất — đề hỏi mốc > 1.000: không
nhóm nào chạm

222 KB

cỡ nhóm lớn nhất — nhỏ hơn ngưỡng 128 MB
~600 lần

Kết luận

Repartition thủ công **không có lợi** ở quy mô này. Nhóm phải cỡ ~250.000 đơn mới đáng bàn. Trả lời "không cần, và đây là số đo chứng minh" ăn điểm hơn trả lời "có, vì đề gợi ý".

Vấn đề thật lại nằm chỗ khác

`spark.sql.shuffle.partitions` mặc định = 200 → 200 task tí hơn cho 69 MB, thời gian đi hết vào lập lịch. Hạ xuống 8-16 hoặc bật AQE coalesce.

Xuất file: chỗ mất điểm lằng xẹt nhất

CSV (bài 1-1, 1-2) — một FILE

Spark ghi ra **thư mục** chứa part-file. Phải `coalesce(1)`, ghi, rồi **đổi tên** `part-*.csv` thành tên đề yêu cầu. Nộp nguyên thư mục = trừ điểm.

Parquet (bài 2-1, 2-2)

Tương tự với `part-*.snappy.parquet`. **Cấm** dùng `getmerge` nối parquet — hỏng footer, file không đọc được.

Kiểm 30 giây trước khi nộp

- ▶ Mở lại từng file bằng pandas, in `shape + head()`
- ▶ 1-1: 3.696 dòng · 1-2: 128 dòng
- ▶ 2-1: cột % toàn 0 + **đoạn chứng minh trong báo cáo**
- ▶ Báo cáo có đủ: giả định đã chốt, bảng shuffle, explain plan

PHÚT 77-80

Ba điều mang về

1. Đọc đề như đọc hợp đồng — mỗi chỗ mập mờ phải chốt và ghi lại.
2. Chi phí của big data nằm ở shuffle, không nằm ở phép tính.
3. Kết quả rỗng không có nghĩa là code sai — chứng minh được thì đó là đáp án.

Secondary sort — cấu hình 4 thành phần

THÀNH PHẦN	CẤU HÌNH
Composite key	(state, d, size)
Partitioner	băm theo (state, d)
Grouping comparator	so sánh (state, d)
Sort comparator	so sánh (state, d, size)

Kết quả: một lần gọi reducer nhận trọn mọi size của một (bang, ngày), đã xếp A→Z. Nếu khoá reduce là cả bộ ba thì mỗi lần gọi chỉ thấy 1 size → không so được → phải thêm job nữa.

```
// reduce((state,d), values đã sort theo size)
best = null
for each size group:
    N  =  $\sum cnt$ 
    m  =  $\sum q / N$ 
    var =  $\sum q^2 / N - m \cdot m$     // chia N
    if N > best.N
        or (N == best.N and var < best.var):
            best = (size, N, var)
// hoà cả N lẫn var → giữ best cũ
// (size đến theo A→Z nên best cũ
//  chính là size nhỏ hơn từ điển)
emit(state, d, best.size)
```


Mảng hiệu — vì sao 2 phiếu thay được 10 phiếu

Cách thường: đơn ngày 05/04 (L=10) phát 10 phiếu "+1"



Mảng hiệu: chỉ 2 phiếu — bên nhận xếp theo ngày rồi cộng dồn



Cộng dồn theo ngày: ... 05/04: 0 →

06/04: 1 · 07/04: 1 · ... · 15/04: 1

→ 16/04: 0 ... (y hết 10 phiếu)

Được: lưu lượng $8,4 \times \rightarrow 2,0 \times$. Trả giá: khoá reduce đổi thành (bang, size) + sort phụ theo ngày; reducer giữ chuỗi 101 ngày (vài KB) trong bộ nhớ.

Con số chi tiết bài 2-1 và 2-2

2-1: bốn cách hiểu "cancelled" — đều ra 0

CÁCH HIỂU	SỐ ĐƠN	≥3 KM
Status chứa cancelled + Standard	6.909	0
Courier = Cancelled + Standard	43	0
Status chứa cancelled, mọi level	18.332	0
Courier = Cancelled, mọi level	5.935	0

2-2: ví dụ lệch mạnh nhất

JNE3567-KR-L · 04/2022 · 120 đơn:

ngưỡng 1,4 vs 1,0 → giữ 12 vs 57 đơn → sd 112,4 vs 54,4.

Một quyết định định nghĩa làm sd lệch **gấp đôi**.

2-2: so sánh P90 / P80

	P90	P80
Nhóm lệch ngưỡng	43,5%	36,7%
Nhóm lệch tập đơn	1,9%	6,1%
Nhóm lệch sd cuối	0,8%	2,4%